AI开发能力复盘报告

一、AI表现出色的领域

1. 快速代码生成与功能原型开发

AI在代码生成和基础功能实现方面表现出极高的效率，尤其适合快速搭建功能原型（MVP）。在本项目中，AI能够在较短时间内生成完整的游戏逻辑与前端结构。

核心游戏逻辑实现

AI能够迅速生成并实现以下关键功能模块：

- 洗牌与发牌逻辑
- 叫地主流程
- 牌型检查与合法性验证
- 基础AI出牌策略

在AI策略方面，初始版本采用了简单的贪心策略，例如：

- 选择能够压过上一家的最小牌
- 在无法压制时选择不出牌

虽然策略较为基础，但能够保证游戏流程完整运行，为后续优化提供了可用基础。

---

2. 前端开发效率极高

AI在 HTML / CSS / JavaScript 前端开发方面表现出非常高的生产效率，可以快速生成完整的 UI 结构和交互逻辑。

在开发过程中，当提出如下需求时，AI通常可以在一次或少数几次迭代内完成实现：

- 调整游戏布局结构
- 添加卡牌选中动画效果
- 实现排序按钮功能

AI能够：

- 直接修改 HTML 结构
- 更新 CSS 布局和样式
- 调整 JS 交互逻辑

3. 支持持续迭代开发

AI在长轮次对话开发中的稳定性表现较好。在本项目中，AI能够支持连续几十轮的需求迭代，包括：

- UI布局调整
- 游戏规则优化
- 组件更新
- 交互逻辑修改

开发流程形成了较为高效的迭代循环：

```
需求提出
↓
AI生成代码
↓
运行测试
↓
发现问题
↓
AI修改代码
↓
再次测试
```

这种循环显著提高了原型开发效率。

4. Debug能力表现良好

当项目出现运行问题时，AI通常能够：

- 快速定位潜在问题代码
- 分析可能的错误原因
- 提出可执行的修复方案

在实际开发过程中，AI可以根据报错信息或运行结果进行推断，并给出修改建议。例如：

- 修改函数调用逻辑
- 调整参数配置

- 或者由开发者手动修正代码

通过 AI 与人工结合，可以形成较为高效的调试流程。

二、AI表现一般的领域

1. 产品体验与用户感知能力不足

AI在功能实现方面表现优秀，但在 **用户体验（User Experience）** 和产品感知方面存在明显不足。

AI更关注：

- 功能是否正确运行
- 逻辑是否符合规则

但较难主动识别以下问题：

- 游戏节奏是否合理
- UI布局是否舒适
- 动画是否自然
- 玩家体验是否流畅

---

2. 体验问题识别能力较弱

在项目开发过程中，以下问题几乎全部由人工发现：

- 斗地主叫分停留时间过短
- UI元素对齐不整齐
- 卡牌显示被裁剪
- 出牌节奏过快

AI在调试过程中主要依赖：

- 页面截图
- 代码逻辑分析

因此 AI 更擅长解决 **功能问题**，但对 **体验问题** 的识别能力较弱。

---

3. UI微调能力有限

AI在UI布局调整方面通常采用 **粗粒度修改方式**。

例如，当发现电脑玩家位置过于靠近屏幕边缘时，AI可能会直接修改CSS中的位置参数或选择一个大致合理的数值。

但这种调整往往缺乏精细优化，仍需要人工进一步微调，例如：

- 精确调整像素距离
- 调整边距
- 平衡整体布局

---

4. 容易产生“看起来正确”的实现

AI生成的代码有时能够满足规则要求，但并不符合真实用户行为。

典型案例：出牌策略

在初始版本中，AI实现的出牌策略为：

> 永远出最小单张

这一策略虽然符合规则、实现简单，但游戏体验较差。例如：

- 即使有对子或三带一
- AI仍然优先出单张

缺乏基本策略判断能力，因此策略设计仍需要人工规划。

---

5. 需求理解可能出现偏差

AI在处理自然语言需求时，可能出现理解偏差。

实际案例：

原始需求

> 出最小牌时，如果有对子就出对子，否则才出单张

AI理解为

> 寻找最小的对子

两者含义不同，因此需要人工进一步解释需求并进行多轮沟通修正。

---

6. 跨语言推理带来的理解偏差

在本项目中，我们使用的是 Google Antigravity 平台，并主要调用国外主流大模型（如 Gemini、Claude）。

在实际观察中发现，当用户使用 **中文输入问题** 时，模型内部推理往往仍以 **英文进行**。

信息链路如下：

```
用户输入（中文）
↓
模型内部推理（英文）
↓
最终输出（中文）
```

在复杂需求或多步骤任务中，这种跨语言转换可能会导致语义细节被弱化，从而增加理解偏差的概率。

如果使用对中文语境优化更好的国产大模型，这类问题可能会得到一定缓解。

三、AI开发过程中的技术反思

1. 回归错误（Regression）

当AI在已有系统中新增功能或修改逻辑时，容易引入新的错误。

例如：

修改出牌接口 → 导致叫地主阶段对话丢失

改进方法：

- 修改代码后进行流程测试
- 检查叫地主
- 发牌流程
- 出牌逻辑
- 游戏结束

确保关键功能未被破坏。

---

2. 硬性逻辑优于Prompt约束

在AI陪玩开发中，如果仅依赖 Prompt 进行限制，模型可能出现 **指令漂移**。

例如：

要求台词不超过20字，但模型仍可能生成更长内容。

更可靠的方法是在代码层进行强制限制，例如：

- Python端截断字符串长度

---

3. Prompt优化之外的系统调优

在模型输出质量不理想时，除了修改Prompt，还可以通过调整模型超参数进行优化，例如：

- `temperature`：控制生成随机性
- `frequency_penalty`：减少重复内容

通过这些参数调整，可以从概率分布层面改善生成效果。

---

4. 多模态能力仍需人工审查

虽然AI能够生成语音（TTS）和界面代码，但以下方面仍需要人工评估：

- 语音自然度
- 音频质量
- UI美观程度

AI缺乏人类的审美能力，因此仍需要人工进行最终质量审核。

四、人类在项目中的不可替代价值

1. 产品经理角色（Product Manager）

负责：

- 游戏整体设计
- UI布局规划
- 交互流程设计

例如：

- 游戏大厅结构
- 玩家区域布局

- 卡牌动画效果

这些涉及用户体验的设计决策，AI难以独立完成。

---

2. 质量控制角色（QA）

人工主要负责发现：

- 游戏节奏问题
- UI显示异常
- 动画体验问题

这些问题往往需要实际体验才能发现。

五、开发流程优化建议

1. UI设计流程优化

推荐流程：

```
AI生成UI草图
↓
人工确认布局
↓
确定组件结构
↓
再进行逻辑开发
```

这样可以减少后期大规模修改。

---

2. 模块化开发

推荐代码结构：

```
rules.js       游戏规则
player.js      玩家逻辑
ai.js          AI策略
ui.js          UI交互
game.js        游戏主控
```

优势：

- 代码结构清晰
- 可读性更高
- 更容易调试

同时在调试阶段可以只提供单个模块代码，从而：

- 减少 context 长度
- 提高AI理解效率

---

3. 提前定义策略逻辑

在开发游戏AI时，不应完全依赖AI自动生成策略。

更好的方法是：

1. 与AI讨论策略方案
2. 明确策略规则
3. 再进行代码实现

这样可以避免AI默认采用过于简单的策略。

总结

通过本项目的实践可以发现：

AI在以下方面表现突出：

- 快速代码生成
- UI开发
- Debug支持
- 快速原型开发

但在以下方面仍需要人工参与：

- 产品体验设计
- 策略规划
- UI精细调整
- 质量控制

因此当前最有效的开发模式仍然是：

> 人类负责产品与策略设计
>
> AI负责高效率代码生成与实现